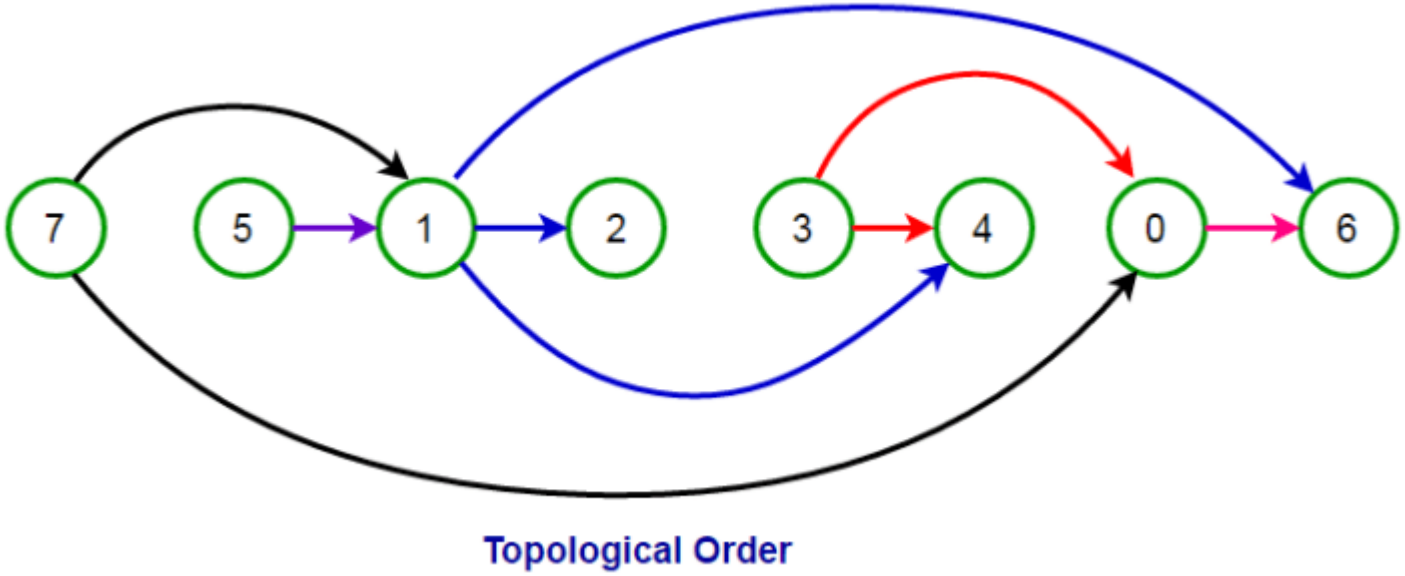
# Topological Sorting

Feasible Linear Ordering of Tasks

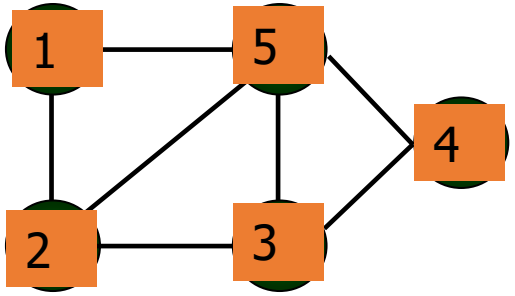


# Remainder : Graph Representations and Traversals

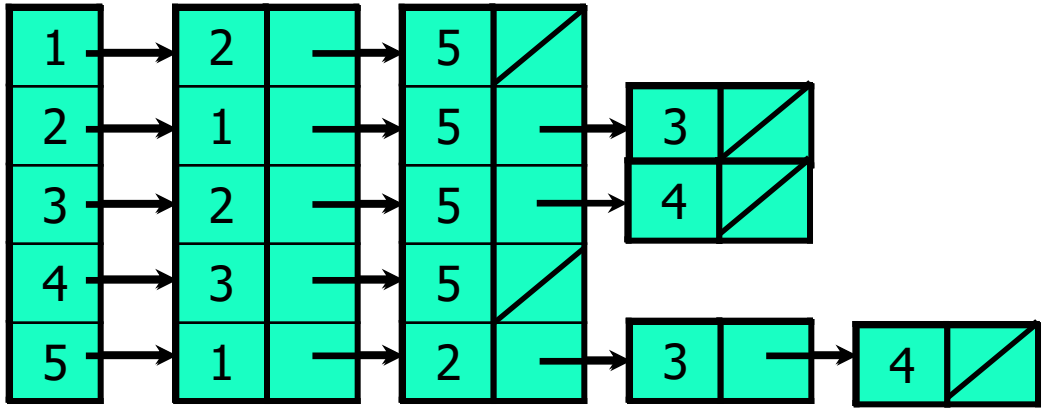
Two ways of representations:

- Adjacency list
- Adjacency matrix

## Adjacency List :

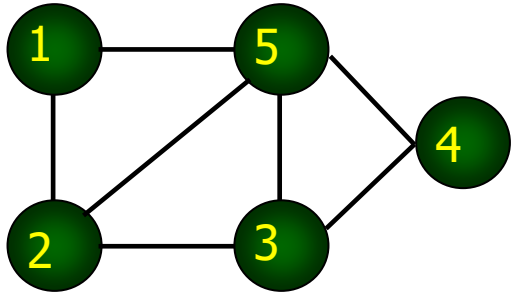



An undirected graph




Adjacency List

# Adjacency matrix




An undirected graph

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| **1** | 0 | 1 | 0 | 0 | 1 |
| **2** | 1 | 0 | 1 | 0 | 1 |
| **3** | 0 | 1 | 0 | 1 | 1 |
| **4** | 0 | 0 | 1 | 0 | 1 |
| **5** | 1 | 1 | 1 | 1 | 0 |

Adjacency Matrix

# Graph Traversals

## Breadth-First Traversal : BFS

- Visit the start and move down horizontally and visit all the nodes of the current level
- Move to the next level and continue until all vertices have been visited.

## Depth-First Traversal : DFS

- Visit the start, travel as deep as possible by visiting all unvisited vertices along the path .
- When the current path ends, backtrack to the predecessor vertex with unvisited neighbors and restart a DFS traversal from there.
- If there are still unvisited vertices , traversal restarts and continues until all vertices have been visited.

# More Graph Definitions

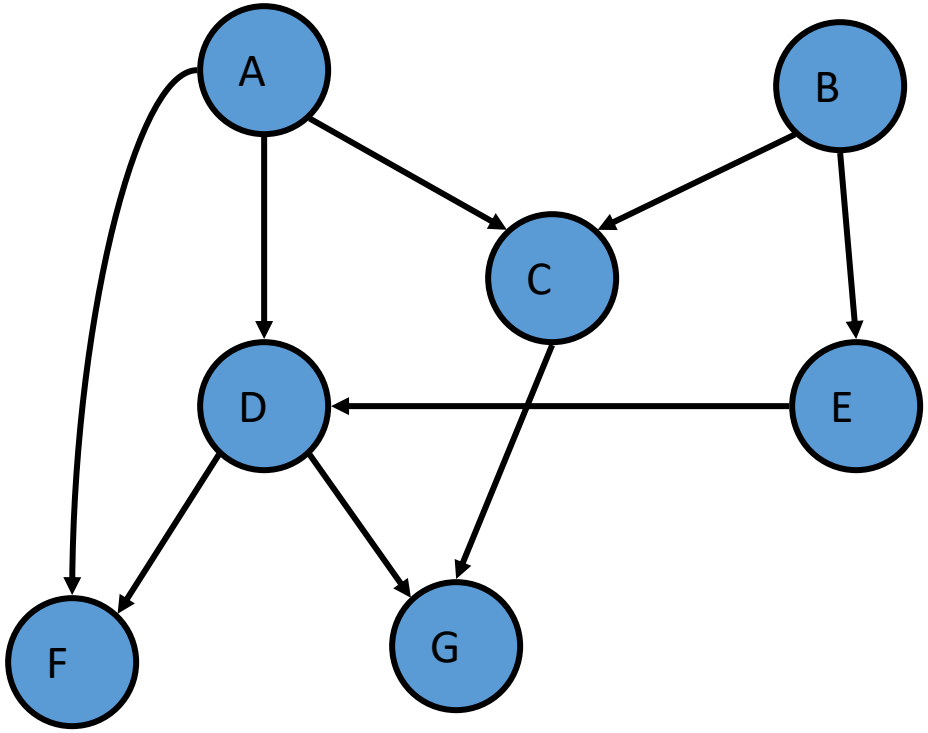
Degree of a vertex U: The number of graph edges which U is the endpoint
→The edges are incident to U.

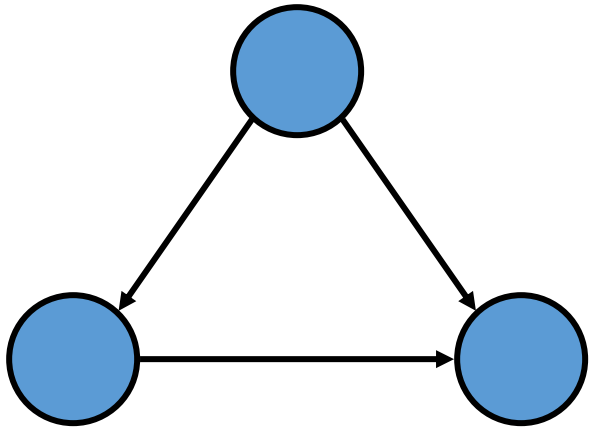
<u>For directed graphs :</u>
Outdegree:The number of outgoing edges from a given vertex U.
Indegree : The number of incoming edges to U

**The algorithms for topological sort uses "indegrees" of vertices.**

# Directed Acyclic Graphs

- A *directed acyclic graph* or *DAG* is a directed graph with no directed cycles:



# Topological Sort : Definition

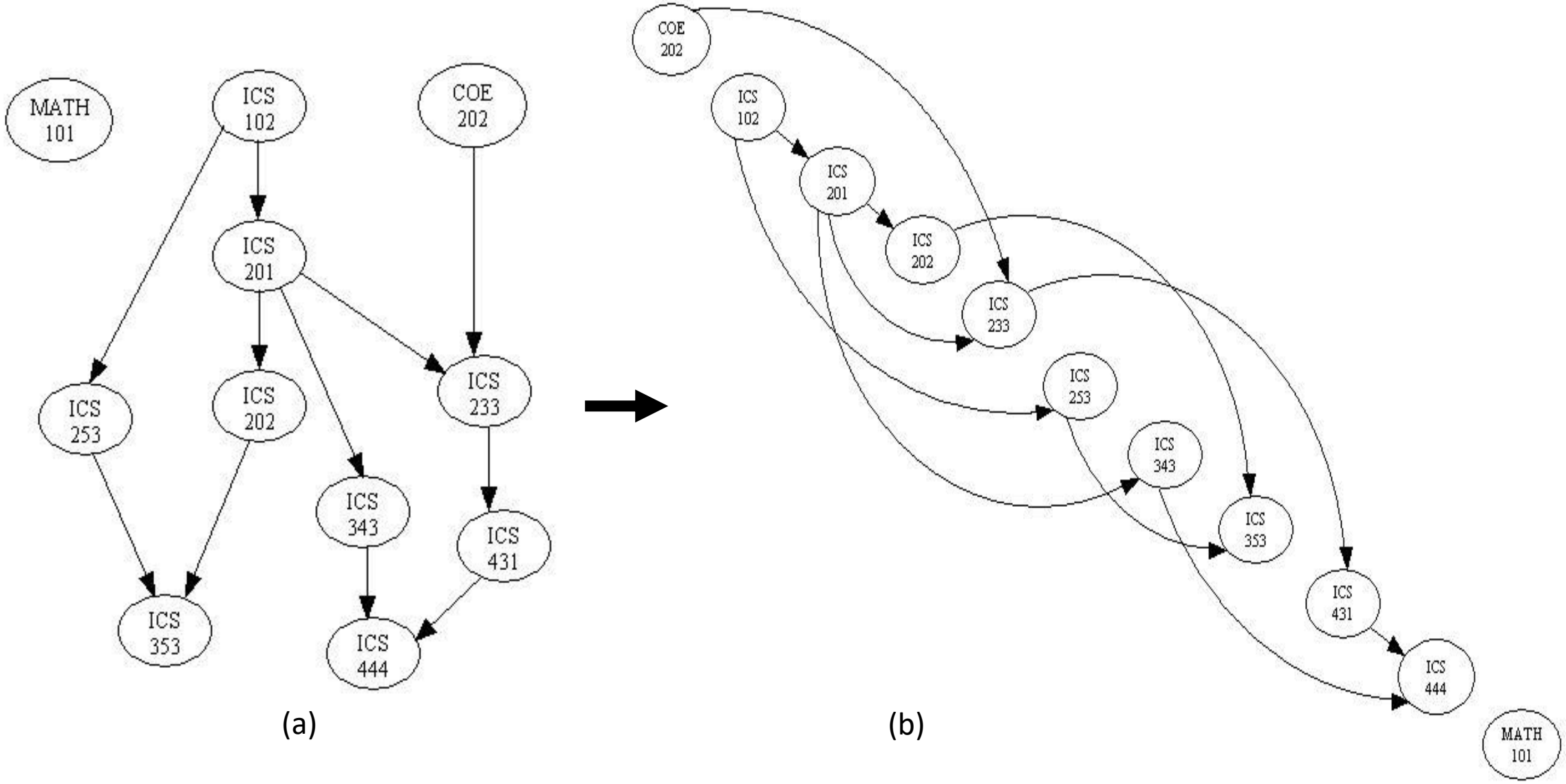
- Suppose we have a set of tasks and a set of dependencies (precedence constraints) of the form: "task A must be done before task B"
- How can we find **an ordering of the tasks that conforms with the given dependencies ?**

**Definition: Topological sort of a DAG is a linear ordering of its vertices such that for every directed edge (u,v) from vertex u to vertex v, u comes before v in the final ordering.**

**Vertices represent the tasks, directed edges represent the precedence constraints**

# Topological Sort Example 1: Course Prerequisite order

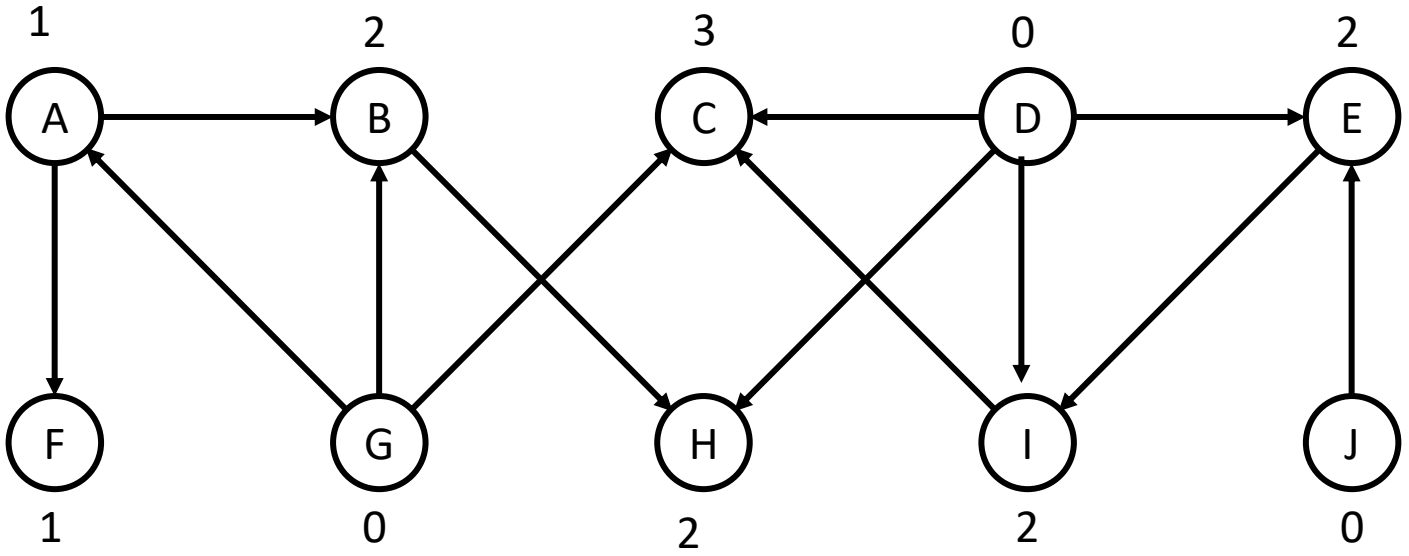
- A curriculum has the following prerequisites in (a):
- The graph in (a) can be topologically sorted as in (b)



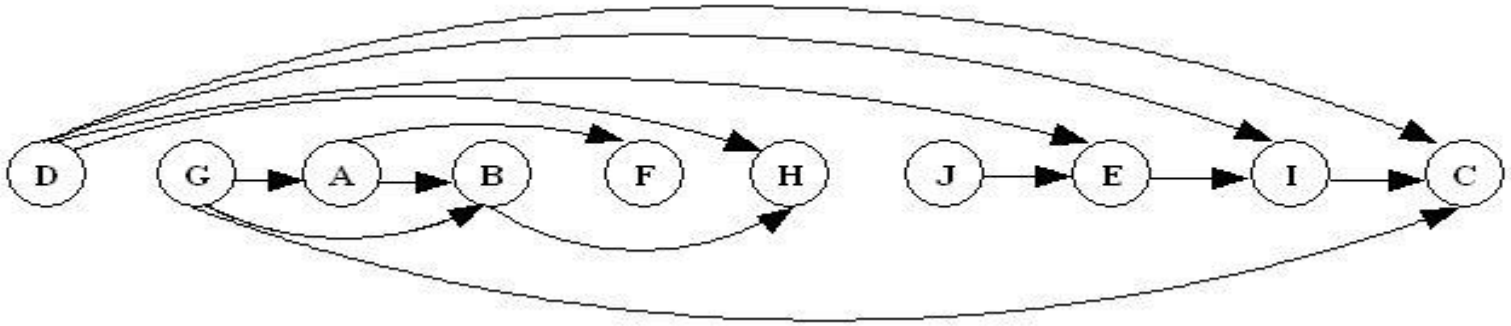

(a) (b)

# Topological Sort Example 2 : Job execution orders

Numbers are indegrees of each vertex



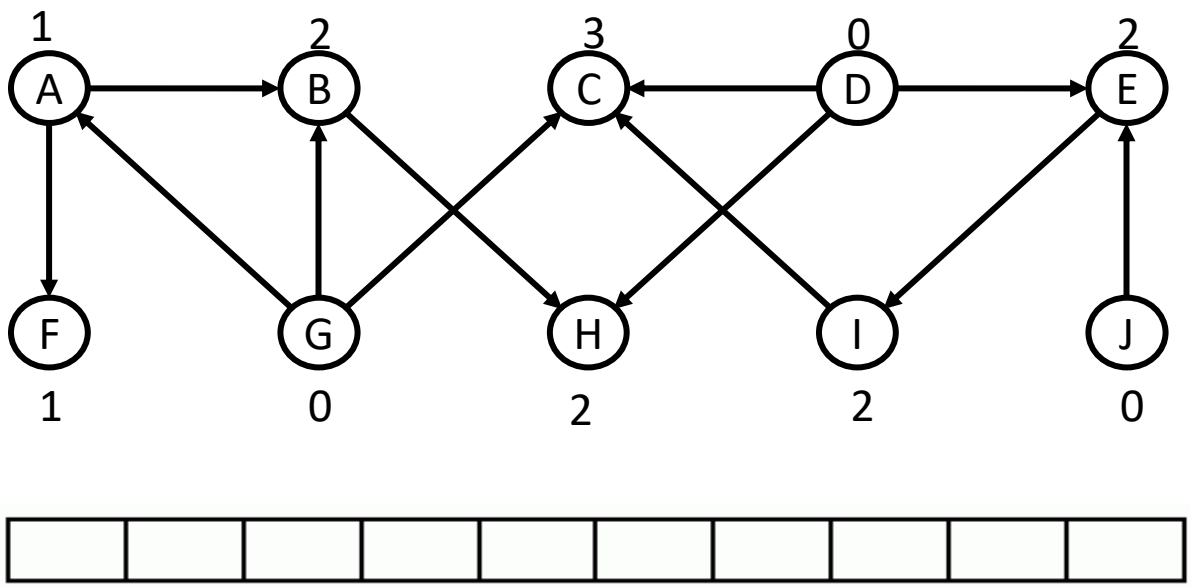

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| D | G | A | B | F | H | J | E | I | C |



# Topological sort more formally

- Is it possible to execute all the tasks in G in an order that respects all the precedence requirements given by the graph edges?
- The answer is "yes" *if and only if* the directed graph **G** has no cycle! otherwise we have a deadlock

→We must have a directed acyclic graph : DAG



# Topological Sort more Formally

- Suppose that we have a DAG

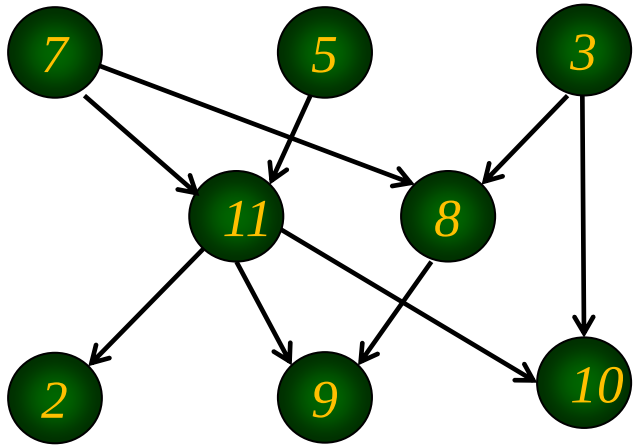
G = (V, E) : vertices **V** represent tasks, and each edge (**u**, **v**)∈**E** means that task **u** must be done before task **v**

- A topological sorting of the vertices in a DAG is an ordering

$$v_1, v_2, v_3, \ldots, v_{|V|}$$

  such that $v_j$ appears before $v_k$ if there is a path from $v_j$ to $v_k$
- There can be multiple topological sorts of G

# Topological Sort : Different Topological Orders




The graph has many valid topological orderings:

7, 5, 3, 11, 8, 2, 9, 10
3, 5, 7, 8, 11,10, 9 ,2
3, 7, 8, 5, 11, 10, 2, 9
5, 7, 3, 8, 11, 10, 9, 2
7, 5, 11, 3, 10, 8, 9, 2
7, 5, 11, 2, 3, 8, 9, 10

Each topological order corresponds to  a feasible schedule.

# Topological Sort Algorithms

- We have a directed acyclic graph (dag) $G=(V, E)$ with n vertices and m edges.
- A topological sort of G is a is linear ordering of all its vertices such that if $G$ contains a directed edge $(u, v)$, then $u$ appears before $v$ in the ordering.
- **In topological sort, vertices are ordered along a horizontal line in such a way that all directed edges go from left to right.**

# Topological Sort Algorithms

1-DFS-based algorithm:

- DFS traversal: Note the order with which the vertices are popped off stack (dead end)
- Reverse order solves topological sorting problem.
- Back edges (Cycles) encountered?→ NOT a DAG!

2-Source removal algorithm

- Repeatedly identify and remove a *source* vertex, ie, a vertex that has no more incoming edges

Topological sort algorithms are greedy algorithms.

# Topological Sort using DFS

Run the DFS on the DAG and output the vertices in reverse order of finishing times.

Execution steps of the algorithm:

1- Declare a linked list: Initially the list is empty

2- Run DFS

3- When a node becomes finished  insert it to the head of the linked list (Use Insfront algorithm)

Note: A stack may also be used in place of the linked list.

# Topological Sort Algorithms-1

Topological-Sort(G,n)

{

Call DFS to compute finish time f[v] for each vertex

As each vertex is finished, insert it onto the front of a linked list

Return the linked list of vertices

}

Basic requirement of DFS is :

$$(u,v) \in G \Rightarrow f[u] > f[v]$$

# Topological Sort Algorithm-1: Example

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

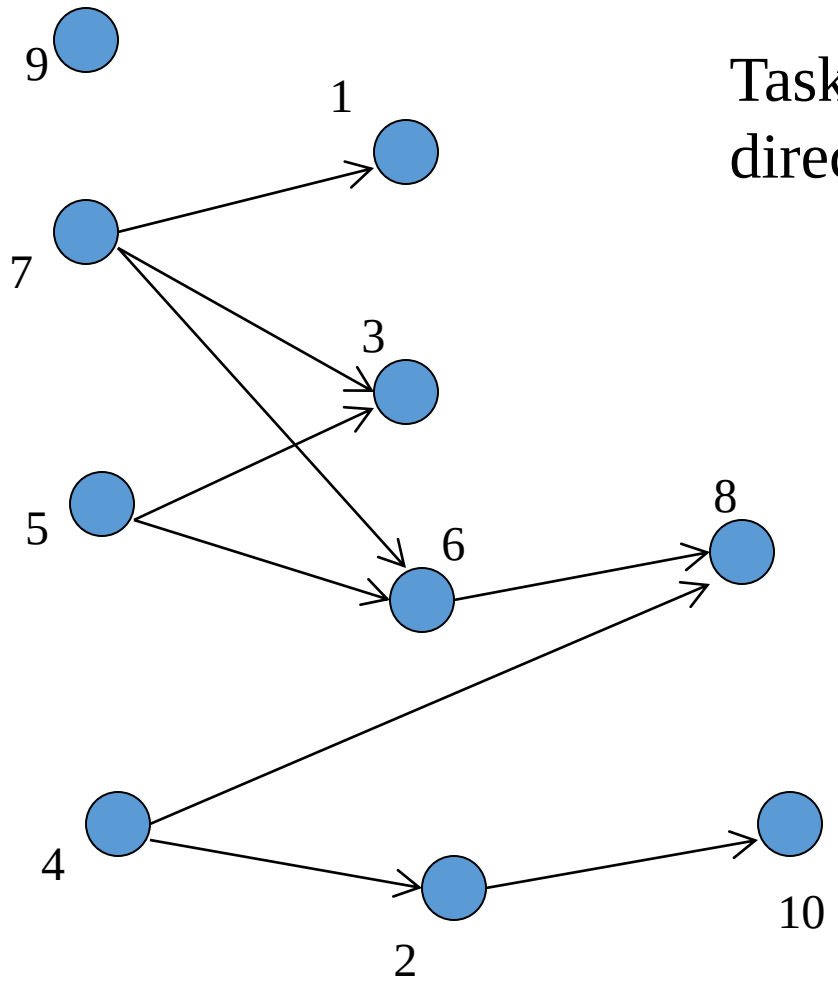
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

Make a directed graph and then do DFS.




Tasks shown as a directed edges.

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

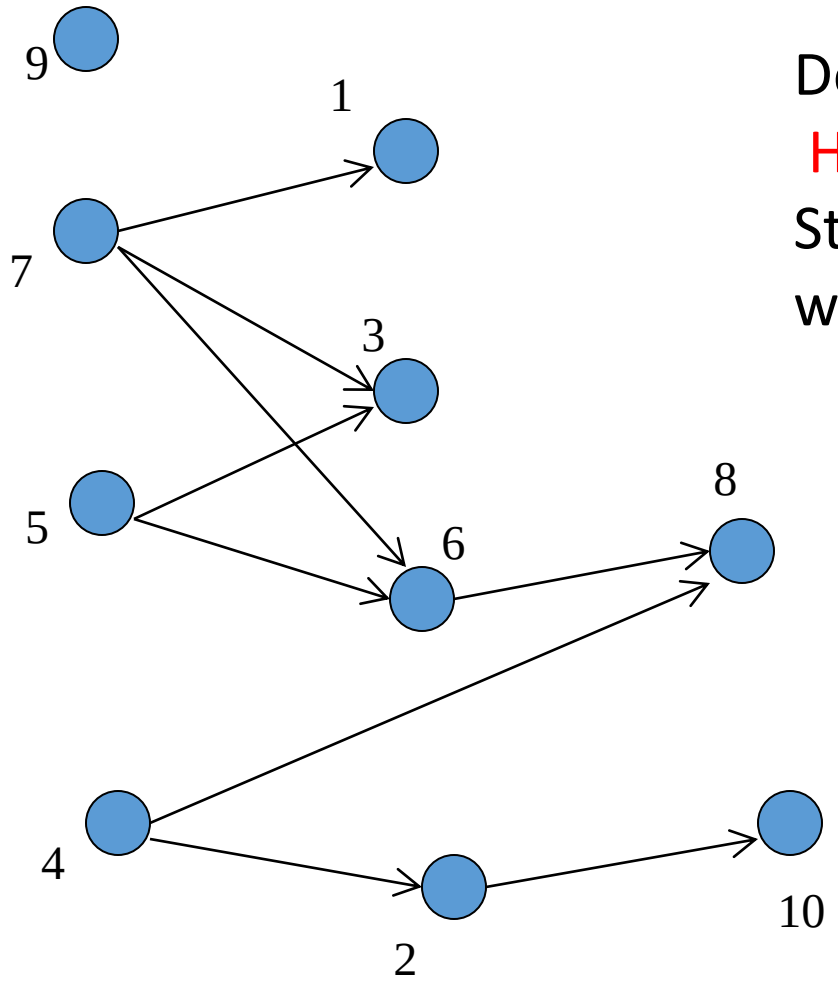
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




Declare Linked List :

Head→Null

Start DFS from a vertex where indegree=0

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

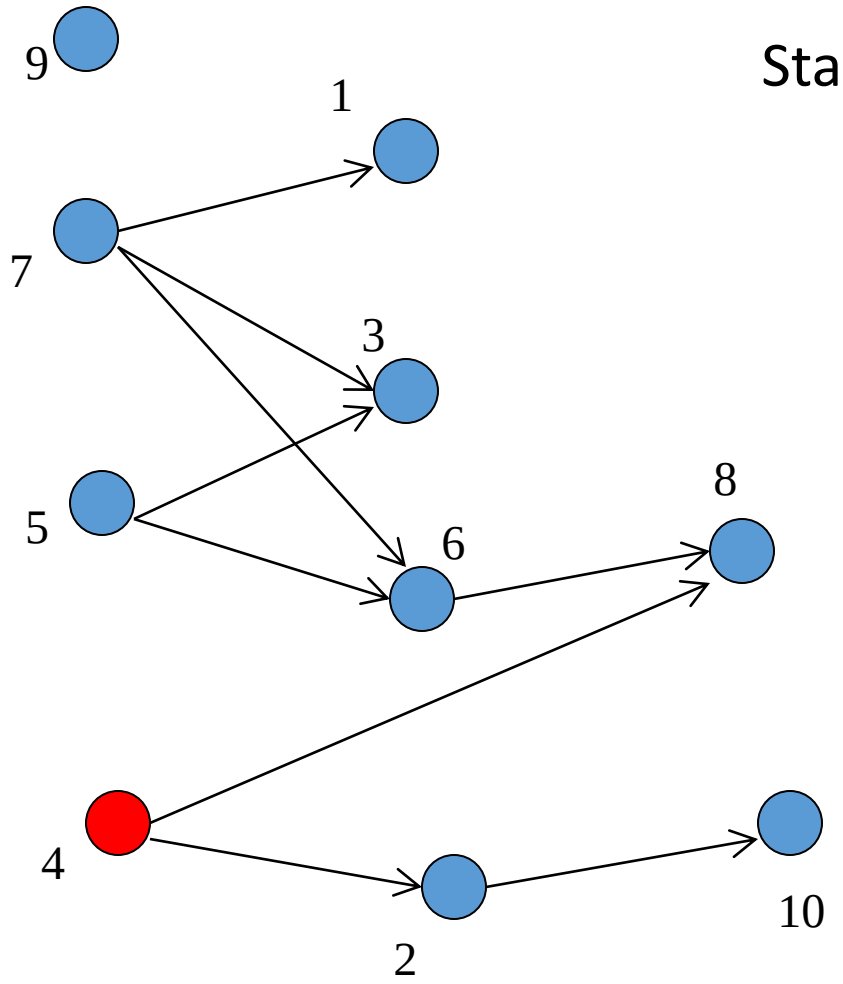
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




Start DFS from vertex 4

A job consists of 10 tasks: 1,2,…,10 .
Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

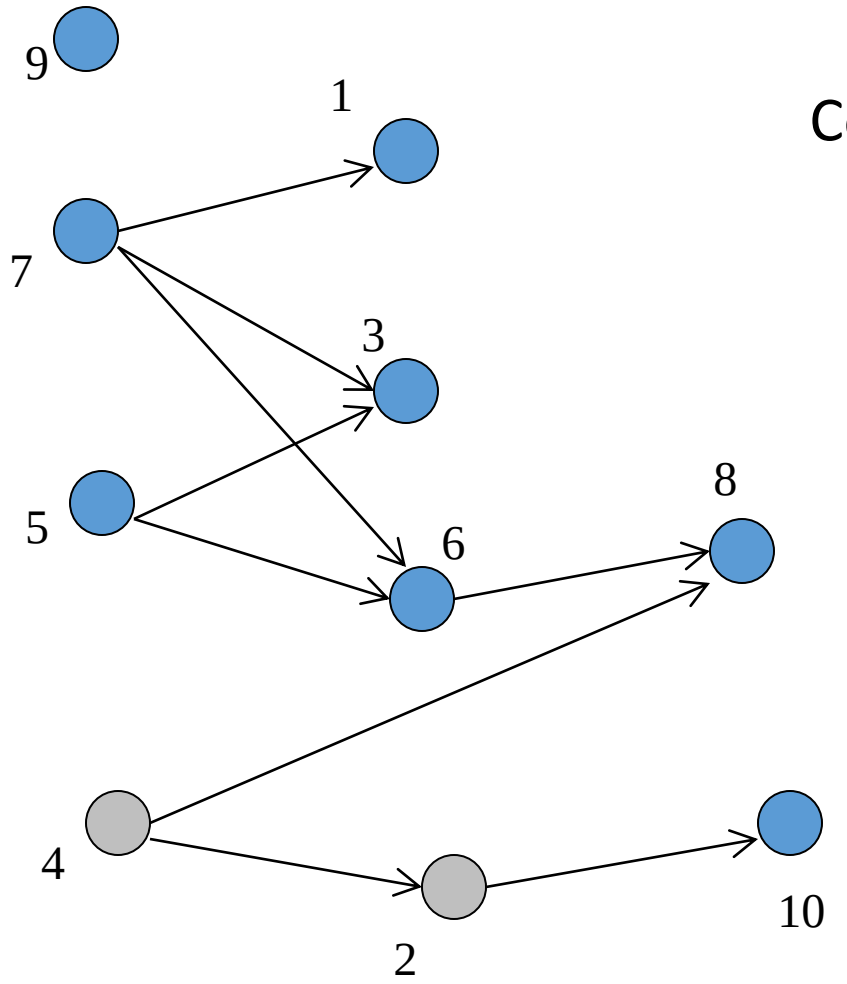
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




Continue DFS until path ends.

A job consists of 10 tasks: 1,2,…,10 .
Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

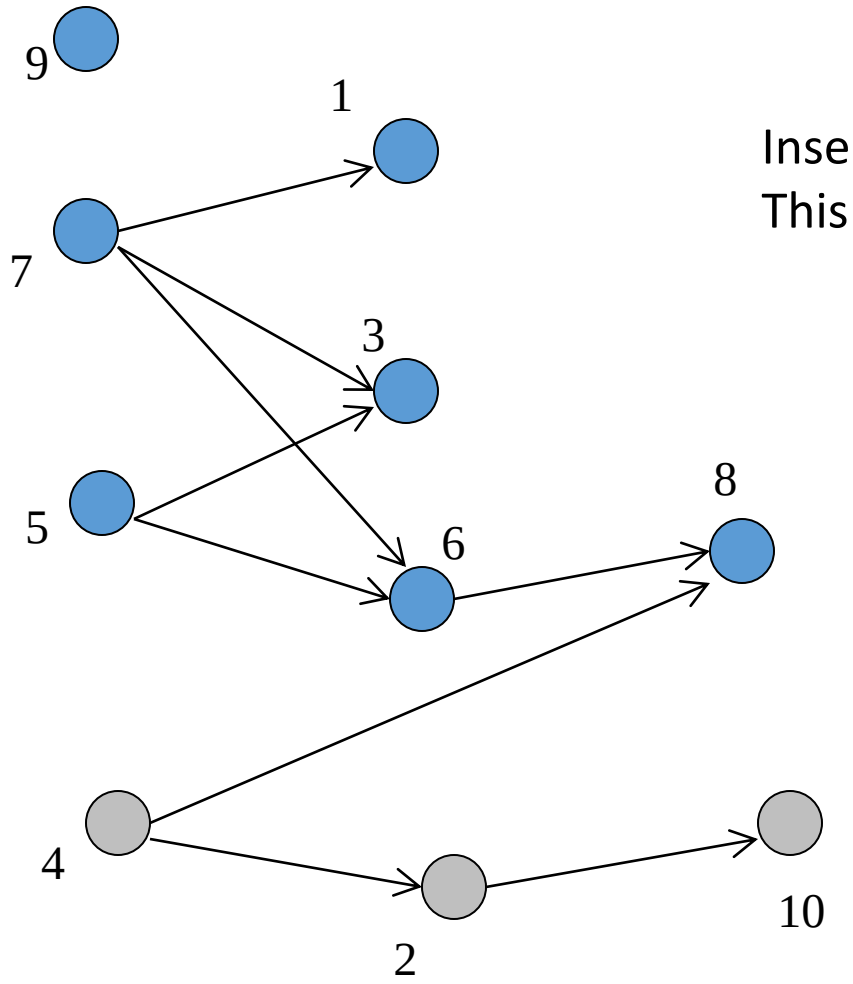
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




Insert the last node (10) to the front of the list
This node becomes finished .

A job consists of 10 tasks: 1,2,…,10 .
Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

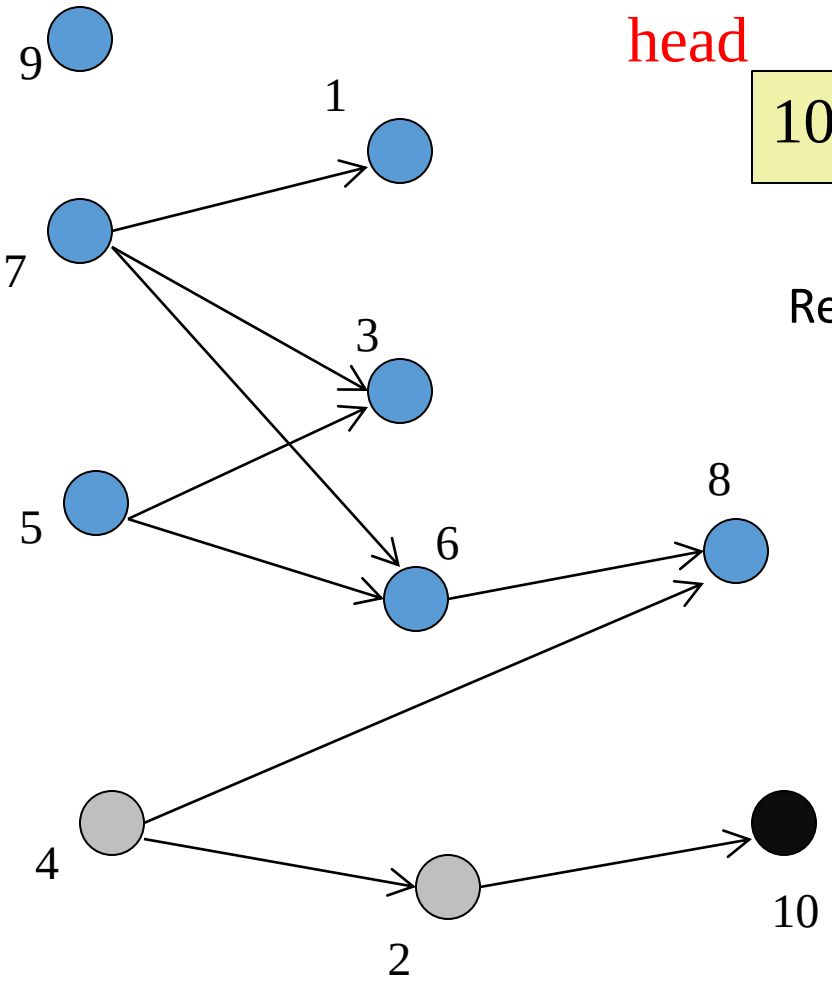
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

| |
|---|
| 10 |

Restart DFS from 4 .

A job consists of 10 tasks: 1,2,…,10 .
Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

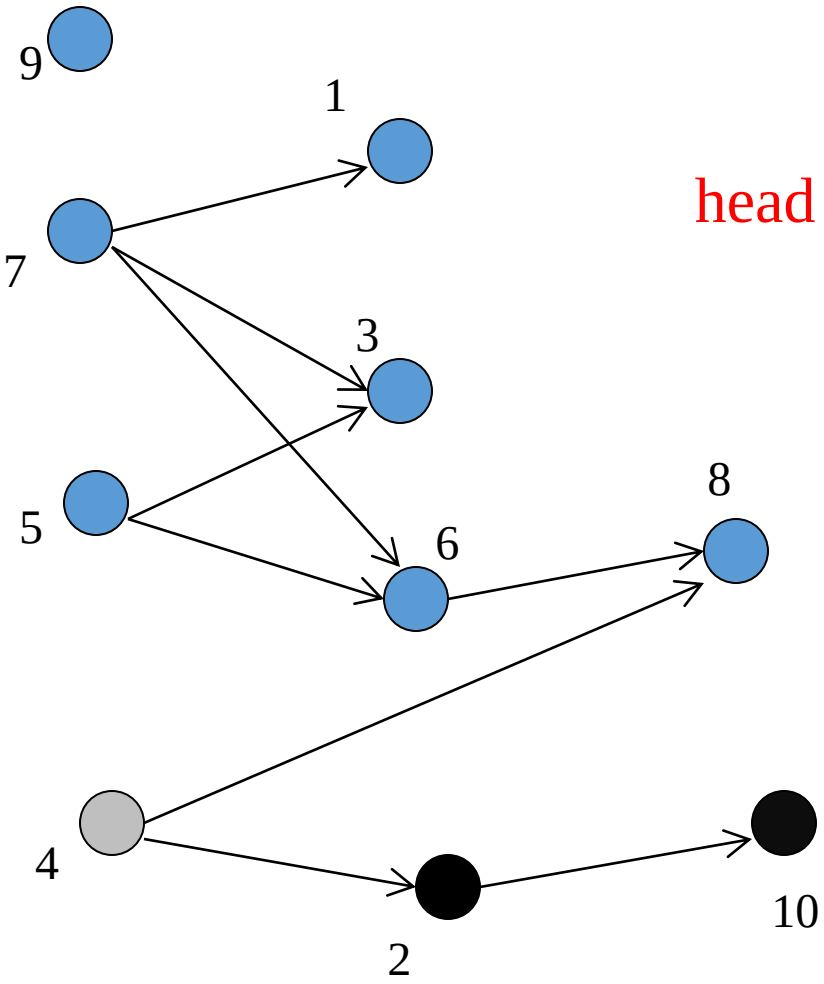
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




Restart DFS from 4, 2 becomes finished
Insert it to the front.

head

2, 10

A job consists of 10 tasks: 1,2,…,10 .
Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

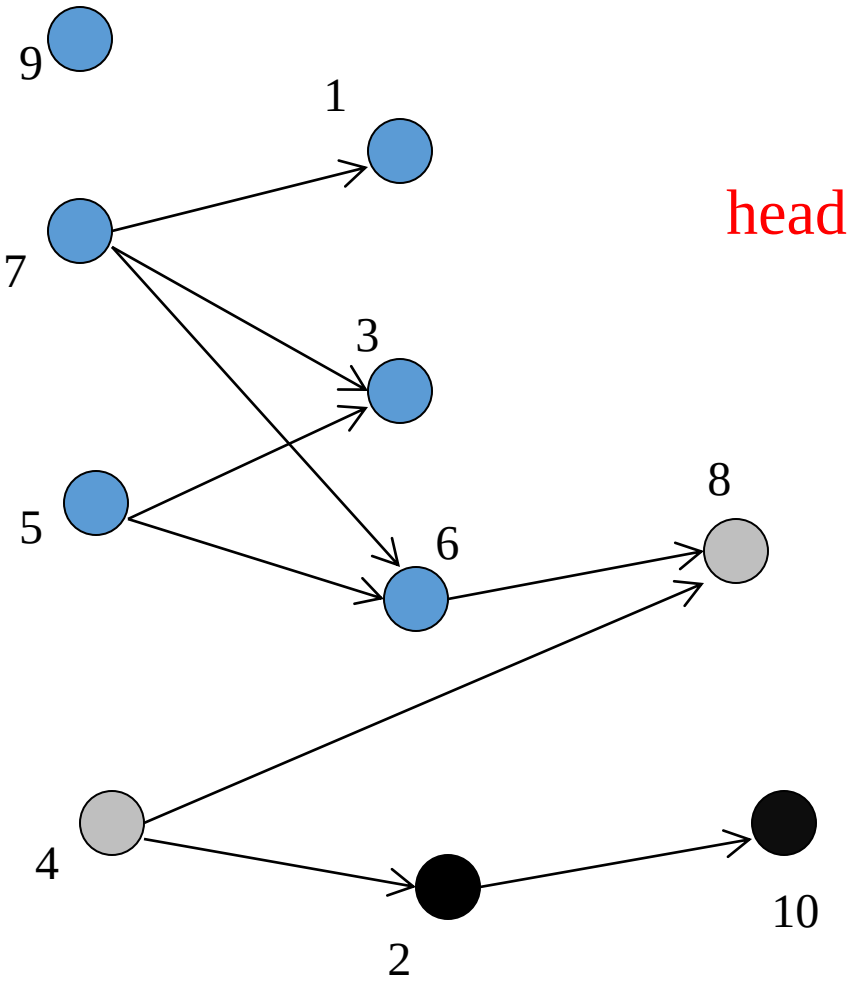
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

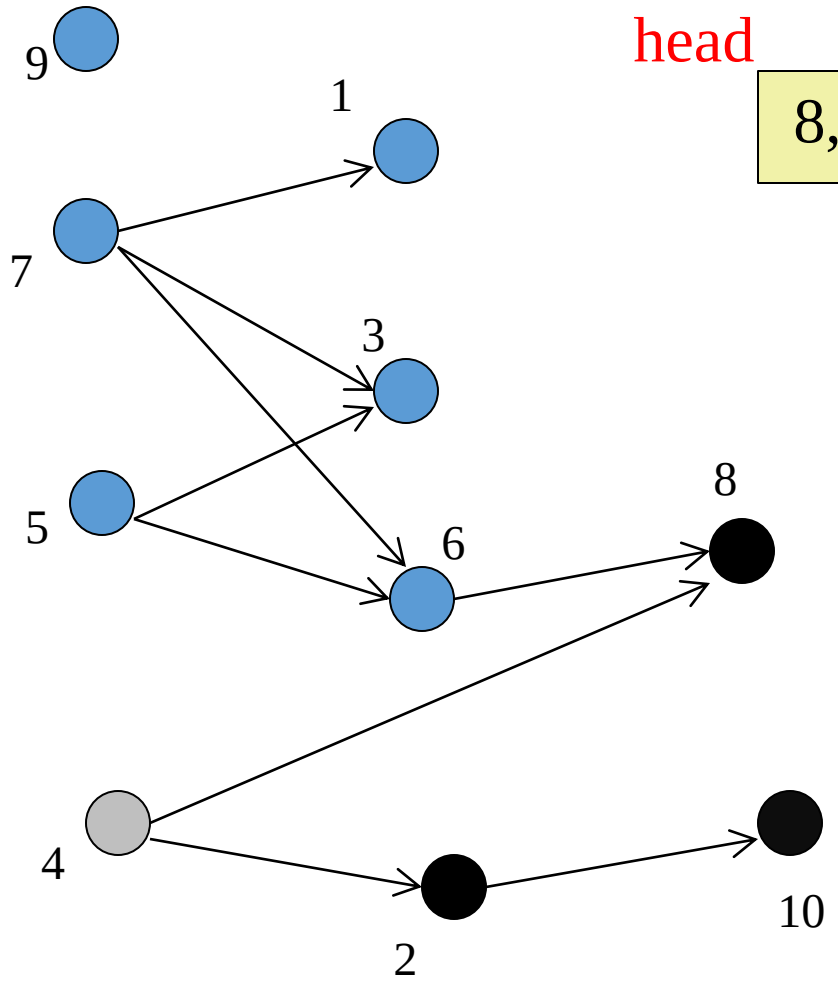
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

8, 2, 10

Continue with remaining nodes.

A job consists of 10 tasks: 1,2,…,10 .
Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

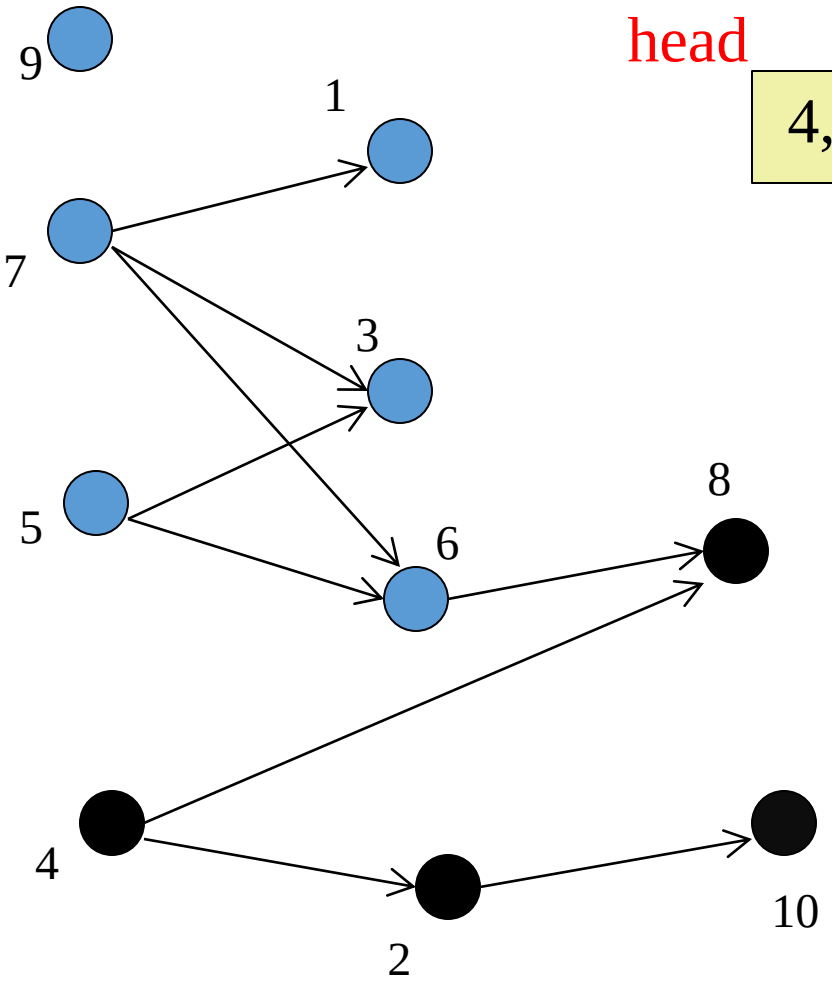
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

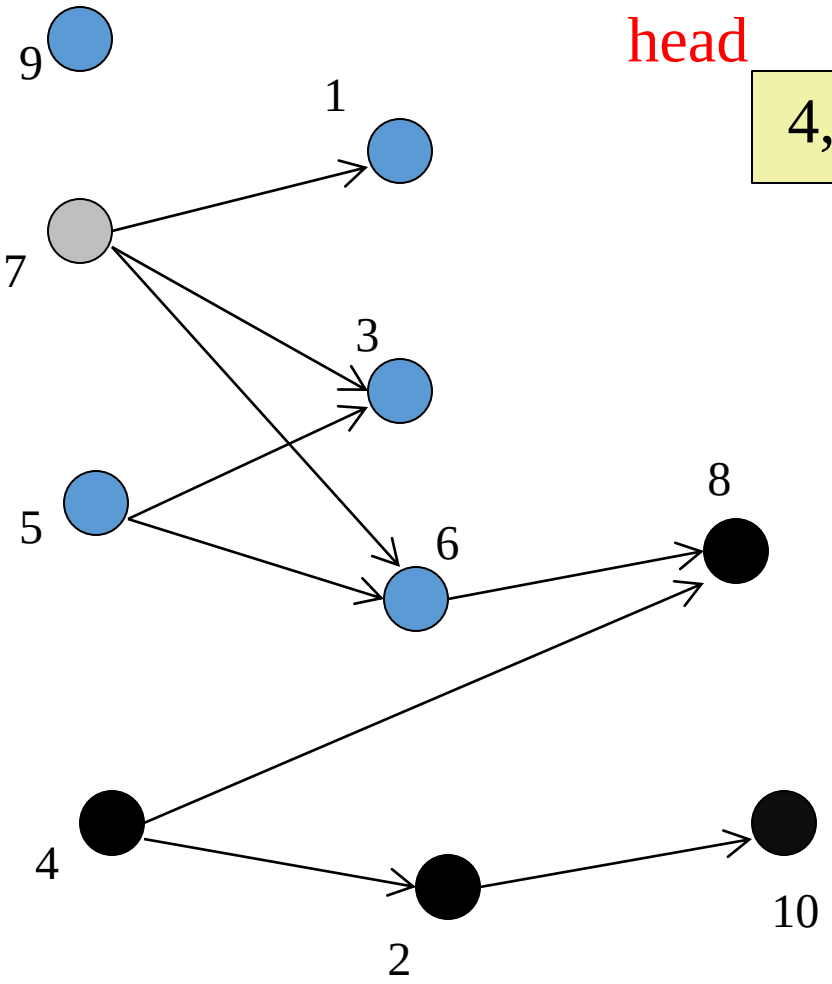
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

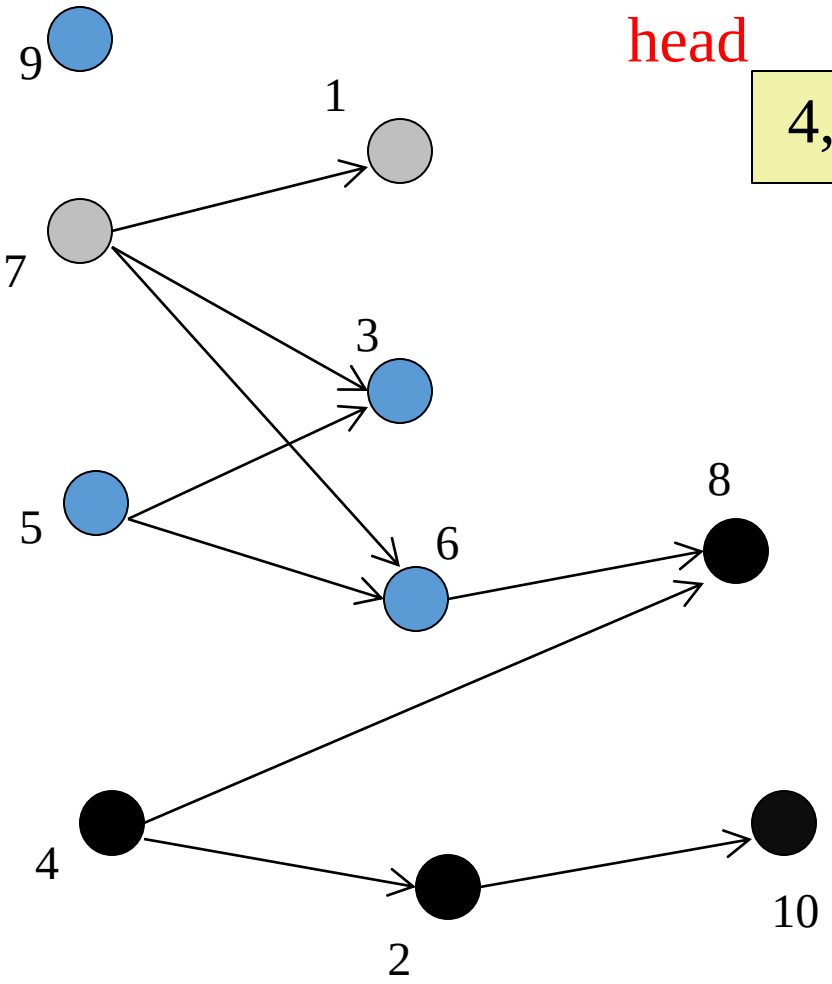
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

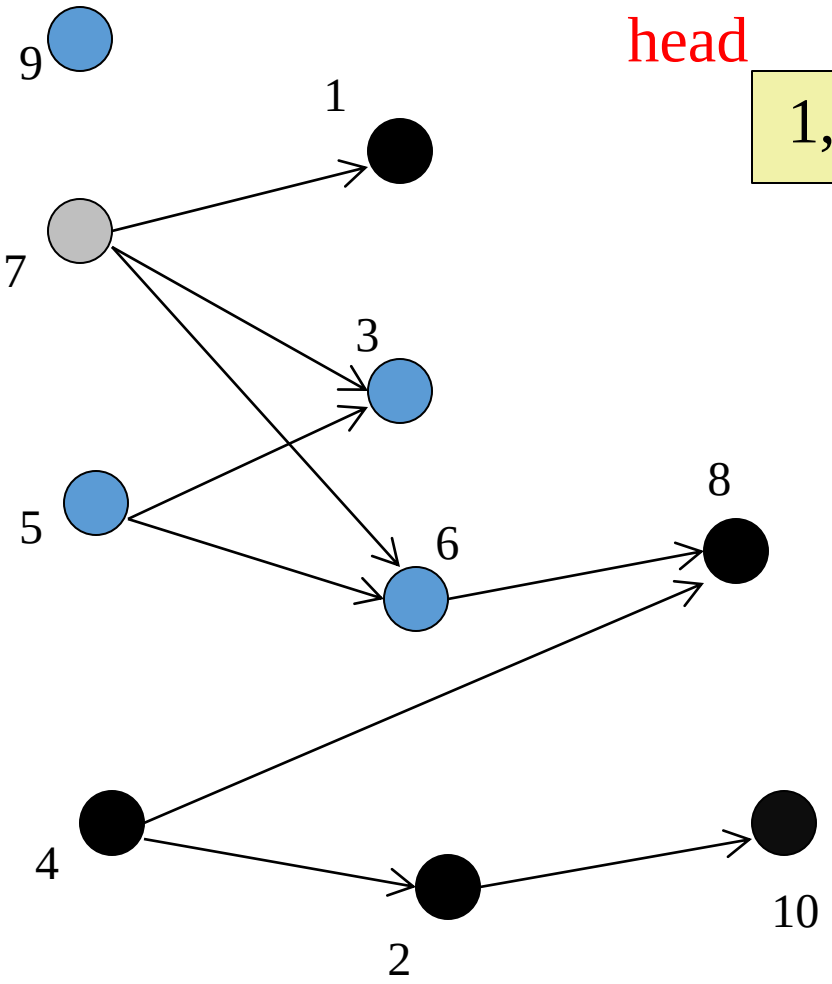
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

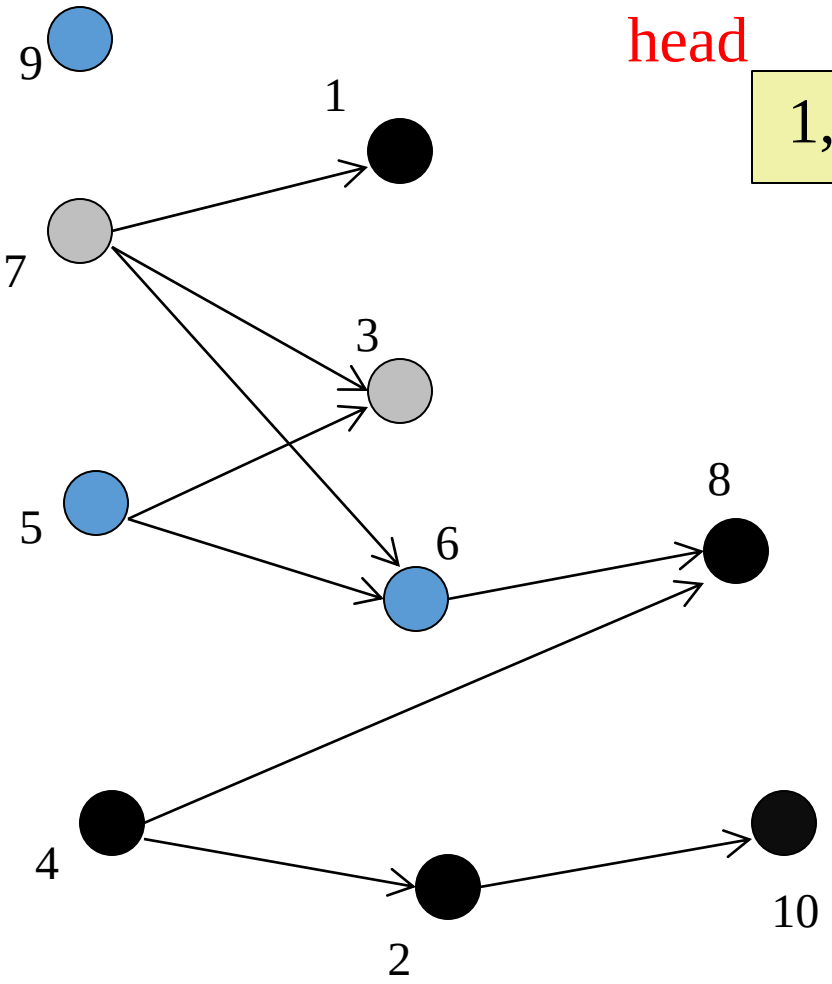
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

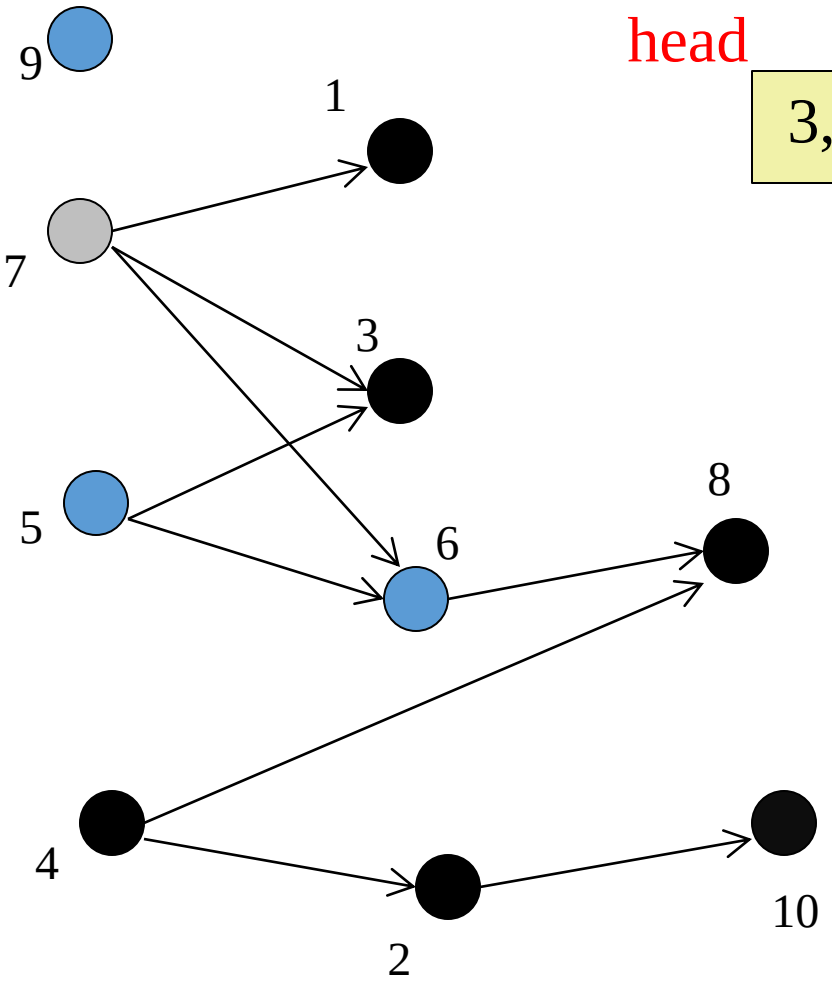
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

3, 1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

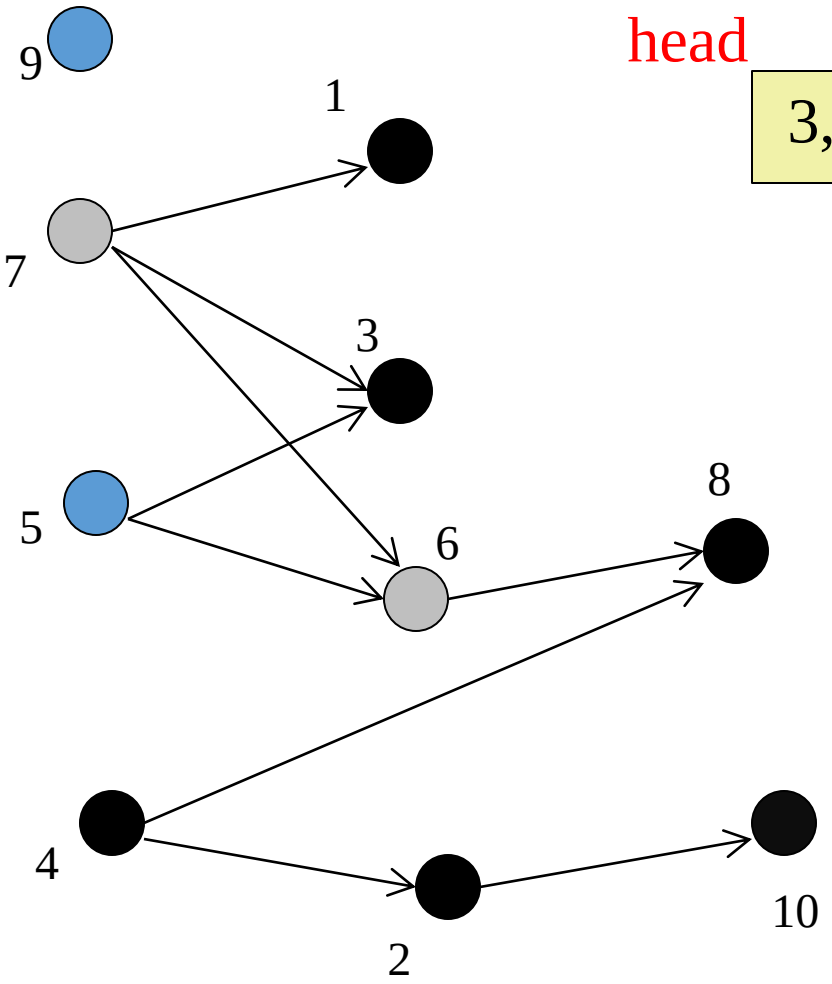
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

3, 1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

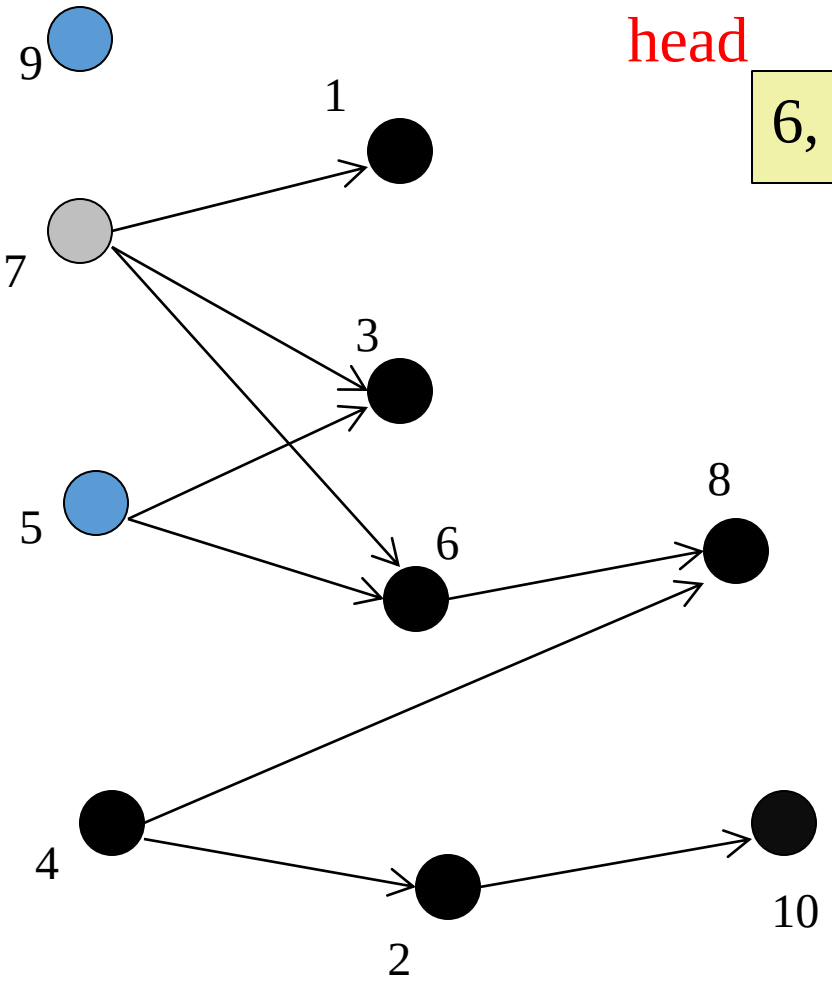
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

6, 3, 1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

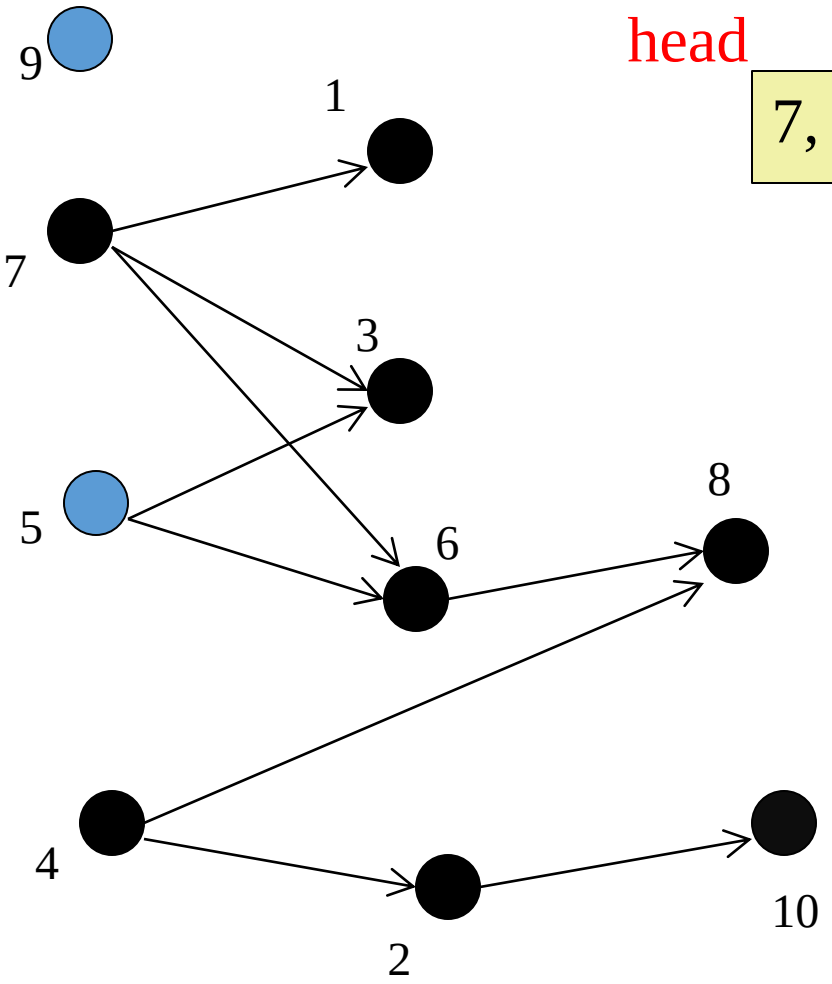
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

7, 6, 3, 1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

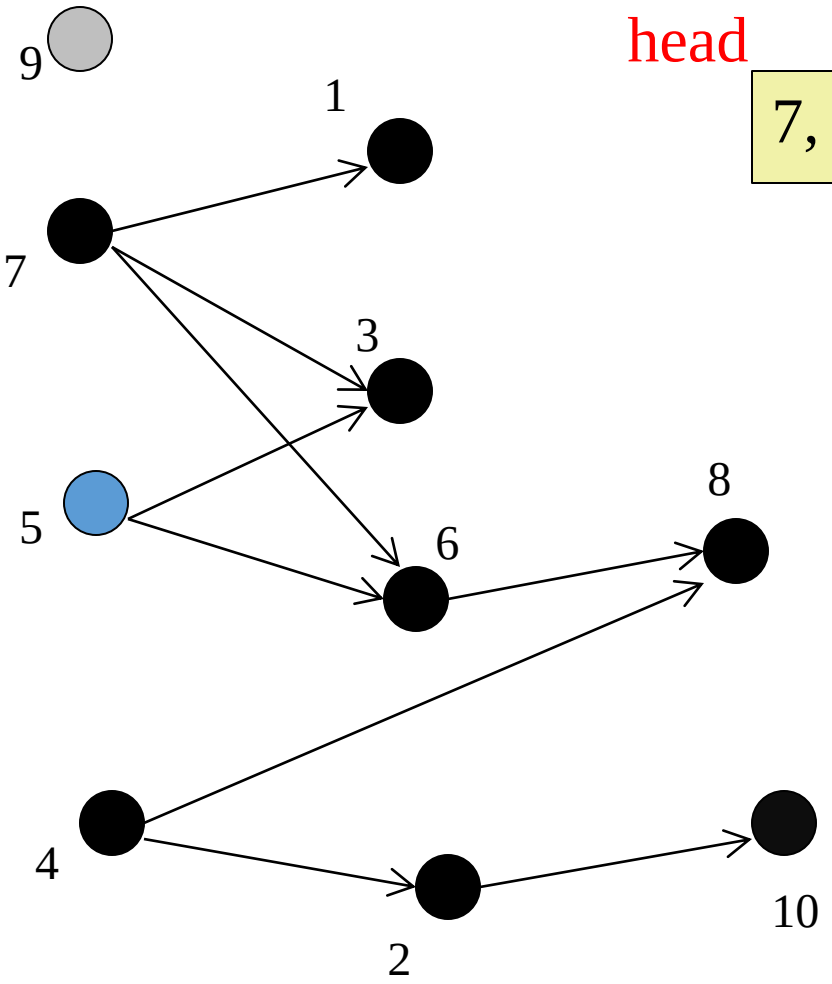
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

7, 6, 3, 1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

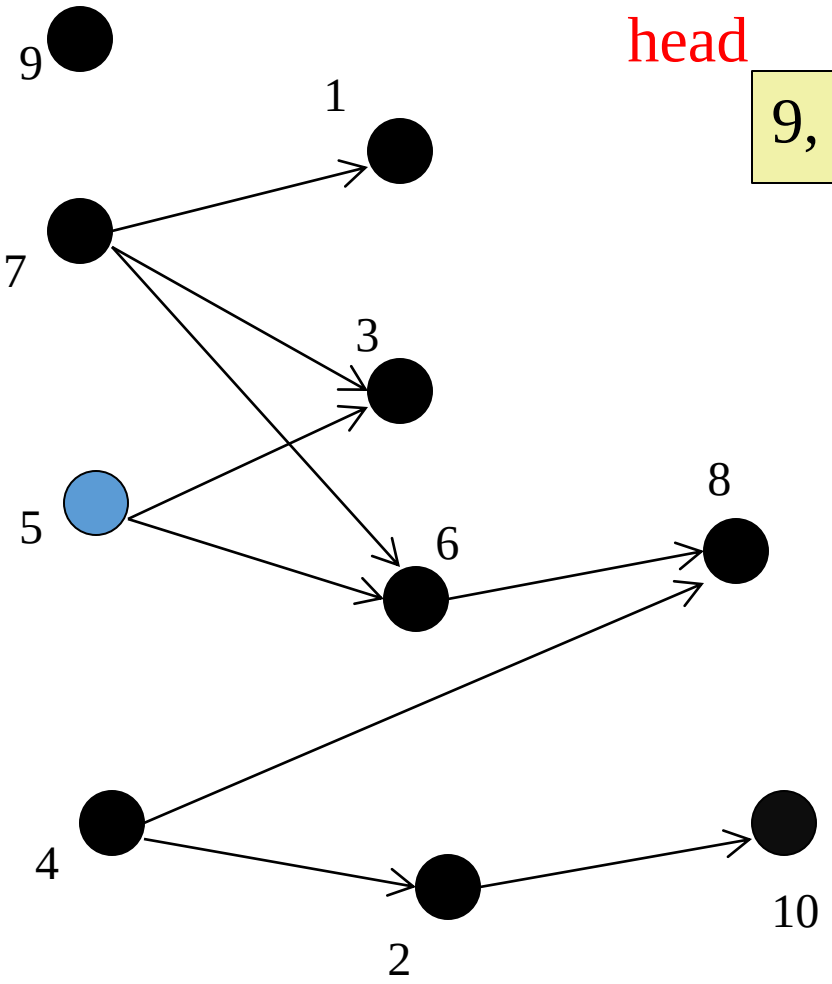
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

9, 7, 6, 3, 1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

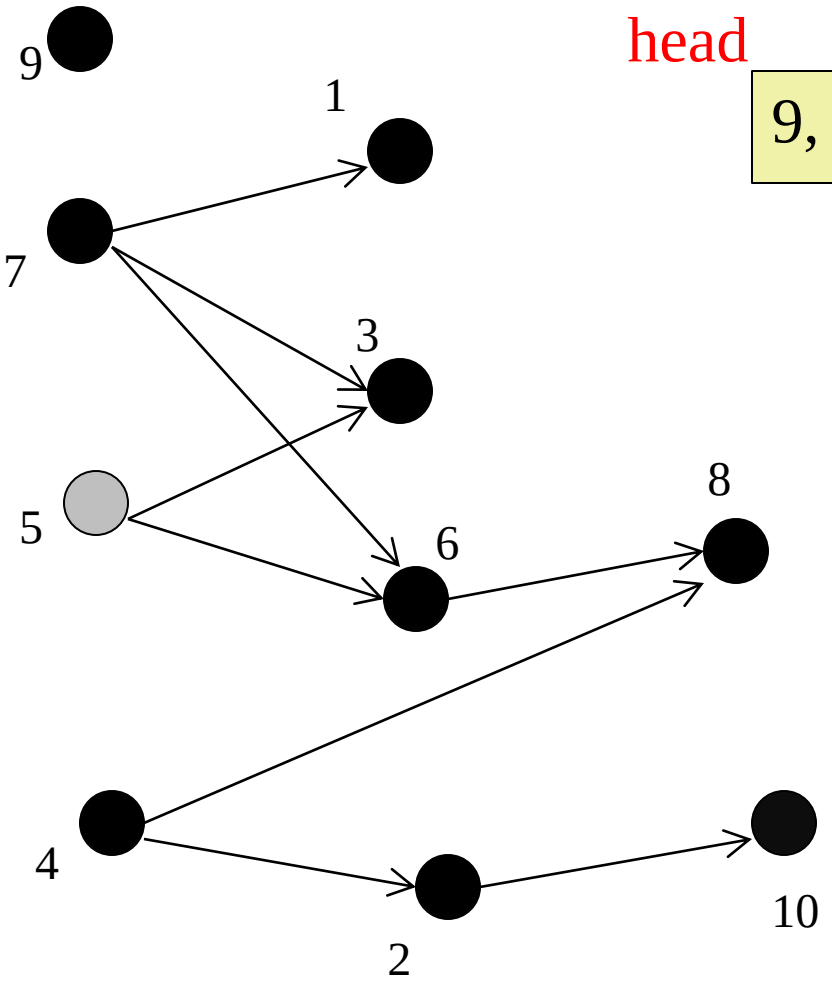
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example




head

9, 7, 6, 3, 1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

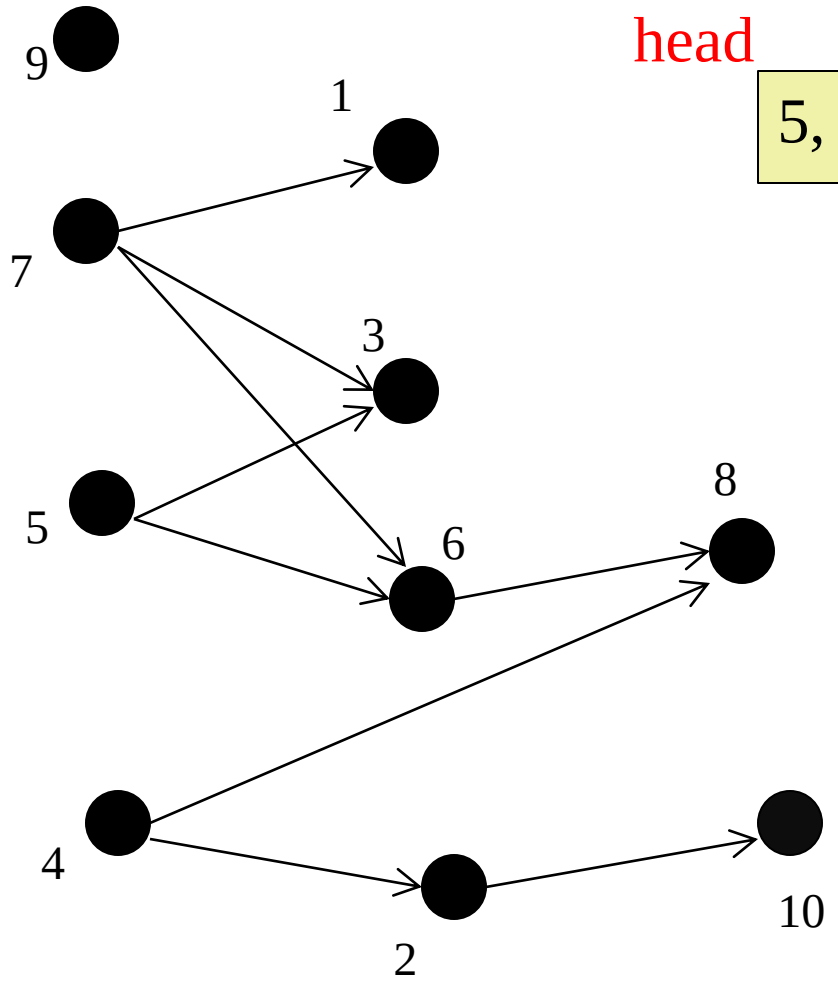
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Example : Final topological order




head

5, 9, 7, 6, 3, 1, 4, 8, 2, 10

A job consists of 10 tasks: 1,2,…,10 .

Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

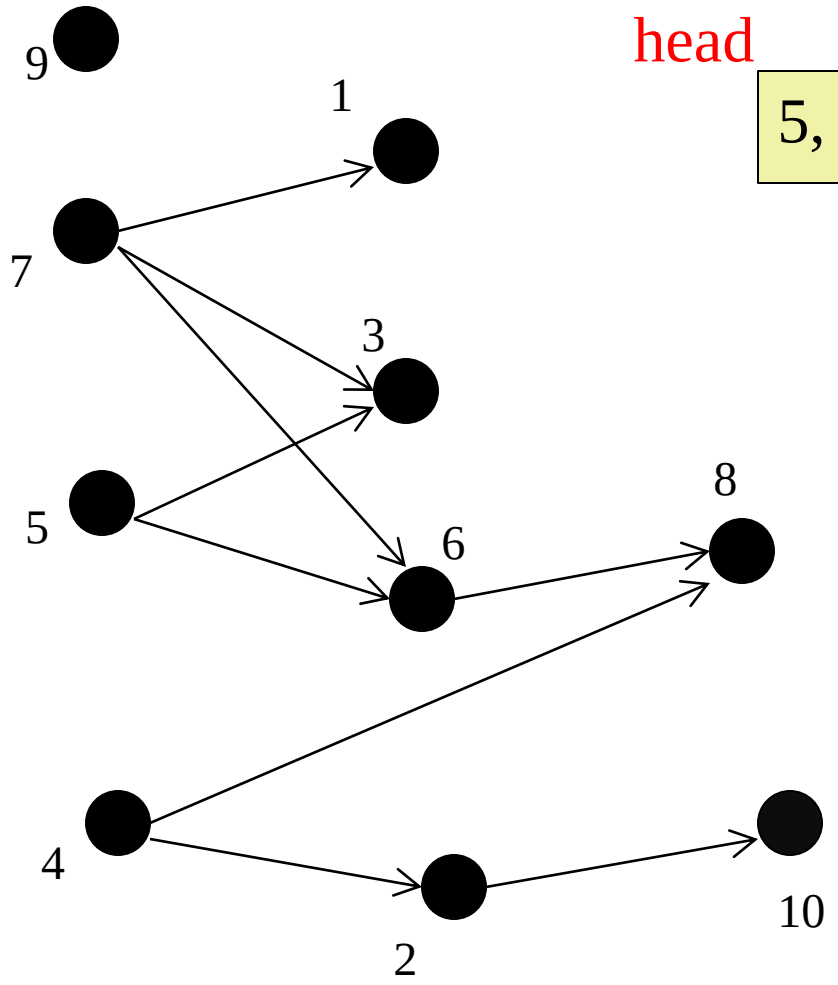
Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Topological Sort: Summary




head

5, 9, 7, 6, 3, 1, 4, 8, 2, 10

The final order of jobs:
Topological order

There can be many orders that meet the requirements

A job consists of 10 tasks: 1,2,…,10 .
Job Precedence rules:

Must start with 7, 5, 4 or 9.

Task 1 must follow 7.

Tasks 3 & 6 must follow both 7 & 5.

8 must follow 6 & 4.

2 must follow 4.

10 must follow 2.

# Time complexity of Topological Sort

- Complexity of DFS based topological sort:
  - Every edge marked at most once: O(|E|) total
  - Every node marked at most once: O(|V|) total
  - So time is linear in |E| and |V|

  The complexity is :

  → **O(n+m) or Θ(n + m)**
  where **n**=|**V**| and **m**=|**E**|
- Why? Depth first search takes **Θ(n + m)** time in the worst case, and inserting into the front of a linked list takes **Θ(1)** time

# Topological Sort Algorithms-2 : Source Removal Algorithm

One way to find a topological sort is to consider in-degrees of the vertices.

Every DAG must have at least one vertex with in-degree zero.

1. Compute the indegrees of all vertices
2. Find a vertex u with indegree 0

   If there is no such vertex then there is a cycle and the vertices cannot be ordered. Stop.

3. Remove u and all its edges (u,v) from the graph.
4. Update the indegrees of the remaining vertices.

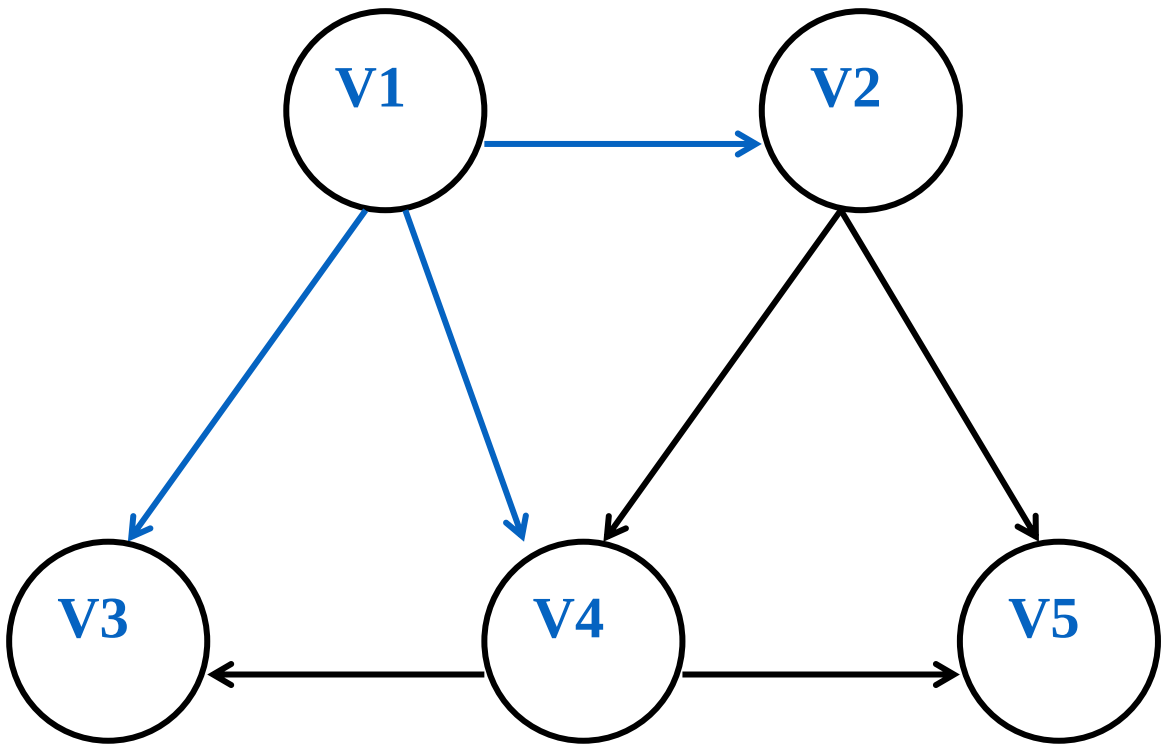
   Repeat steps 2 through 4 while there are vertices to be processed.

# Topological Sort : Source Removal Algorithm

```
Algorithm topologicalOrder(G,n)
    int numVisitedVertices = 0;
    Compute and store indegrees in an array a
    while(numVisitedVertices <= n){
        if(there is no vertex with in-degree 0)
             break
        else
       {select a vertex v that has in-degree 0
        visit v
        numVisitedVertices++
        add v and all its originating edges to
     output list
        update a
       }
     }
```

# Example

| Indegrees | |
|---|---|
| V1 | 0 |
| V2 | 1 |
| V3 | 2 |
| V4 | 2 |
| V5 | 2 |




**Start Vertex : V1**

**Remove it and its edges**

**Update indegrees.**

**New indegrees:**

| | |
|---|---|
| V2 | 0 |
| V3 | 1 |
| V4 | 1 |
| V5 | 2 |

**Remove v2 and V5.**

Possible topological sorts: V1, V2, V4, V3, V5 or V1,V4,V3,V2,V5

# Complexity

- Computing indegrees : O( |V| )
- To find a vertex of indegree 0 we scan all the vertices: |V| operations.
- We repeat this in updates: $|V|^2$ operations

→ Complexity = $O(|V| + |V|^2) = O(|V|^2)$

- This can be reduced to O(log |V| |V| ),using a minheap to store indegree array a.

# Application Areas

- Topological Sorting is mainly used for scheduling jobs from the given dependencies among jobs.
- Computer science applications include:
  - Instruction scheduling,
  - Ordering of formula cell evaluation when recomputing formula values in spreadsheets
  - Determining the order of tasks in compilers
  - Resolving dependencies.